Introducing

"A Java Library that keeps LLMs
on "Need to Know" basis."

Table of Contents

The What, the Why and the How

PrOOPt is an open-source Java library that applies Object-Oriented principles to LLM prompt engineering.

Instead of handing your entire process to a frontier model and hoping for the best, PrOOPt lets you declare — at the function level — exactly where AI has authority and where deterministic code takes over.

***

Think of it as dependency injection for intelligence. You wire the model into specific annotated functions, not into your entire application. Every `@PromptFunction` is a governance decision. Every `@CodeFunction` is a line the LLM cannot cross. Every orchestrator plan is auditable from the first step to the last.

The philosophy is simple: LLMs should reason about problems, not compute answers. When you forbid computation, you force genuine reasoning.

***

Built for Java 17+. Ships with an embedded ONNX Runtime for local inference. Zero external processes. One Maven dependency. MIT licence.

Because great ideas often have humble beginnings.

@PromptFunction

Prompt as SOLID Functions

An LLM-backed method that follows SOLID principles. Write the method signature, declare the prompt template, and return null. PrOOPt intercepts the call via Spring AOP, enriches the prompt with format instructions, routes it to the declared model tier, and autoboxes the LLM's response into whatever Java type you declared.

Be it LocalDate, a POJO or an enum, your return type is the contract. PrOOPt honours it.

***

Each function carries its own model tier — LOCAL, CLOUD_FAST, CLOUD_ADVANCED, or AUTO. That's per-function governance, not per-application guesswork.

```
@PromptFunction(
    model = ModelTier.LOCAL,
    prompt = "Extract the contract date: {text}",
    description = "Extracts signing date"
)
public LocalDate extractDate(String text) {
    text = text.trim(); // optional pre-processing
    return null;        // PrOOPt takes over
}
```

}

@CodeFunction

Deterministic Abstraction, Tooling for LLM invocation.

Marks a method as deterministic Java. The method has a full code implementation and is registered in the orchestrator's tool registry alongside `@PromptFunction` methods. The LLM knows it exists. It can invoke it. It can never replace it.

Use `@CodeFunction` for math, validation, formatting, business rules, and everything deterministic. The compiler doesn't hallucinate. Neither does Math.pow().

Every `@CodeFunction` is a knowledge injection. It extends the LLM's effective capability beyond its training boundary. Your proprietary risk model, your internal scoring formula, your novel algorithm — none of these exist in any training corpus. `@CodeFunction` makes them available as first-class tools the orchestrator can reason about and invoke.

```
@CodeFunction(
    description = "Computes compound interest"
)
public static double compoundInterest(
       double principal, double rate, int years) {
    return principal * Math.pow(1 + rate, years);
}
```

@PromptOrchestrator

Facilitates LLM Reasoning, not Prediction.

The autonomous agent brain. It receives your system prompt and a registry of all discovered `@PromptFunction` and `@CodeFunction` tools, then generates a DAG execution plan — deciding which functions to call, in what order, and which to run in parallel. Independent subexpressions fire concurrently. Cross-stream dependencies resolve non-blocking via CompletableFuture.

The LLM reasons about the problem structure. It does not compute. It cannot compute. That constraint is not a limitation — it is the architecture of reliable intelligence. The orchestrator thinks. The `@CodeFunction` executes. The audit log proves it.

```
@PromptOrchestrator(
    prompt = "You are a legal analyst.",
    planMode = PlanMode.DYNAMIC,
    model = ModelTier.CLOUD_ADVANCED,
    parallel = true
)
@PromptFunctionScan
public class LegalAnalyzer
        extends BaseOrchestrator {
    public static void main(String[] args) {
        PrOOPt.run(LegalAnalyzer.class, args);
    }
}
```

Use Case: Proprietary Trade Secrets Encapsulation

Your bank's internal credit scoring formula was developed over twenty years of actuarial data. It has never been published. It exists in no training corpus, no research paper, no Stack Overflow answer. No LLM — regardless of parameter count or training budget — can compute it correctly. Ask one to try and it will confidently produce a number. Just not the right one.

PrOOPt solves this by encoding the formula as a `@CodeFunction` that the orchestrator invokes as a tool. The LLM reasons about when to apply the scoring model — it reads the loan application, classifies the risk tier, and decides the formula is needed. The `@CodeFunction` computes it deterministically. Twenty years of institutional knowledge, encapsulated in a static Java method, executed without a single token.

Proprietary knowledge stays proprietary. The LLM never sees the formula — only the result. The audit log records exactly which tool was called, with what inputs, and what it returned. Your trade secret is used but never exposed. That's the `@CodeFunction` guarantee.

Use Case: Novel Mathematical Problems Outside LLM Training Set.

Ask an LLM to solve a system of linear equations and it will confidently produce an answer — occasionally the right one. The problem is not knowledge; the model has read every textbook ever written. The problem is that interpolation degrades under precision requirements. When statistical prediction is close enough, the answer looks right. When it isn't, the answer looks equally confident but is wrong.

PrOOPt forces the LLM to reason from first principles: identify the problem type, select the solution strategy, generate a DAG execution plan. Gaussian elimination runs as a `@CodeFunction` — deterministic, reproducible, and provably correct every single time. The LLM planned the solution. Java computed it. The result is guaranteed by the compiler, not by probability.

This is what first-principles forcing means in practice - The LLM cannot recall a memorised answer because the computation is structurally forbidden. It must understand the problem well enough to decompose it into steps it can delegate. Understanding becomes mandatory. That is not a limitation. That is better engineering.

Use Case: PII Masking & Data Sovereignty

When a customer's loan application hits your pipeline, the Social Insurance Number, date of birth, and account number must never leave your network perimeter. Not sometimes. Not with encryption. Never. That is not a feature request. That is a regulatory obligation.

PrOOPt's ModelTier.LOCAL routes sensitive classification and extraction to an embedded ONNX model running entirely inside the JVM — zero cloud calls, zero data in transit, zero third-party processors. The `@CodeFunction` handles PII masking deterministically before any data reaches the CLOUD tier for higher-order reasoning. By the time the frontier model sees the request, every sensitive field is already redacted.

Data sovereignty is not a configuration option you toggle in a YAML file. It is a per-function annotation. `@PromptFunction(model = ModelTier.LOCAL)` means this data does not leave the JVM. Period. The developer declared it. The annotation enforces it. The audit log proves it. Your compliance officer can read the code and verify the boundary.

PrOOPt doesn't ask you to trust the framework with your data. It asks you to read three annotations and verify for yourself.